Amazon Product Co-purchasing Network Analysis

Tharit Thongwandee
DS210 Final Project

A. Project Overview

Goal

This project analyzes the Amazon product co-purchasing network to uncover patterns in how products are related based on customer purchasing behavior. The key questions addressed are:

1. How do product communities form in the network, and do they align with formal product categories?
2. Which product categories show the strongest cross-category purchasing relationships?
3. Which products are most central to the network, serving as connectors between different communities?
4. What is the overall structure and density of the co-purchasing network?

Dataset

The dataset used is the Amazon product co-purchasing network metadata from Stanford's SNAP collection (https://snap.stanford.edu/data/amazon-meta.html). It contains information about approximately 548,552 products including their ASINs, titles, categories, and co-purchasing relationships ("Customers Who Bought This Item Also Bought"). For this analysis, we worked with a sample of 10,000 products representing 5 major categories and containing 30,051 co-purchasing connections.

B. Data Processing

Loading the Data

The Amazon metadata is provided in a non-standard text format that required custom parsing logic. The data loading process involves:

1. Reading the file line-by-line using Rust's `BufReader`
2. Parsing each product entry and its hierarchical attributes
3. Extracting co-purchasing relationships between products

For development and testing efficiency, I implemented sampling functionality that allows working with a manageable subset of the data:

```
pub fn load_sample_data(file_path: &str, sample_size: usize) ->
Result<Vec<Product>, Box<dyn Error>> {
```

```rust
    let file = File::open(file_path)?;
    let reader = BufReader::new(file);

    let all_products = parse_amazon_meta(reader)?;

    // Take a random sample using Rust's iterator random sampling
    let sampled: Vec<Product> = all_products
        .into_iter()
        .choose_multiple(&mut thread_rng(), sample_size);

    Ok(sampled)
}
```

Data Cleaning and Transformation

The parser handles several data quality issues:

- Missing or incomplete product information (titles, prices, etc.)
- Products without category information
- Invalid ASINs in co-purchasing relationships
- Inconsistent category formatting
- Removal of duplicate connections

C. Code Structure

1. Modules

- **main.rs**: Entry point that coordinates the overall workflow, processes command-line arguments, and orchestrates the analysis pipeline.
- **data.rs**: Handles data loading and preprocessing, containing parser logic for the Amazon metadata format.
- **graph.rs**: Implements the graph representation using petgraph, with functions to build and manipulate the network.
- **analysis.rs**: Contains all network analysis algorithms including metrics calculation, community detection, and centrality measures.
- **visualization.rs**: Handles output formatting and saving results to CSV and JSON files for further analysis.

This modular organization separates concerns and provides a clear flow from data loading to analysis to results presentation.

2. Key Functions & Types

Product Struct (data.rs)

```rust
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Product {
    pub asin: String,
    pub title: Option<String>,
    pub categories: Vec<Vec<String>>,
    pub price: Option<f64>,
    pub sales_rank: Option<u64>,
    pub avg_rating: Option<f64>,
    pub review_count: usize,
    pub similar: Vec<String>,
}
```

- **Purpose**: Represents an Amazon product with all its metadata.
- **Usage**: Core data structure for storing product information throughout the analysis pipeline.

build_graph (graph.rs)

```rust
pub fn build_graph(products: &[Product]) -> Result<NetworkGraph, Box<dyn Error>>
```

- **Purpose**: Constructs the co-purchasing network from product data.
- **Inputs**: Array of Product structs
- **Outputs**: NetworkGraph (a petgraph undirected graph)
- **Logic**: First adds all products as nodes, then adds edges representing co-purchasing relationships.

detect_communities (analysis.rs)

```rust
pub fn detect_communities(graph: &NetworkGraph, num_communities: usize, min_community_size: usize) -> Vec<Community>
```

- **Purpose**: Identifies communities of related products in the graph.
- **Inputs**: The network graph and parameters controlling community detection
- **Outputs**: Vector of Community structs containing grouped products
- **Logic**: Uses connected components analysis and community detection algorithms to group related products.

find_densest_subgraph (analysis.rs)

```rust
pub fn find_densest_subgraph(graph: &NetworkGraph) -> Vec<String>
```

- **Purpose**: Finds the 2-approximation of the densest subgraph.
- **Inputs**: The network graph
- **Outputs**: Vector of product ASINs in the densest subgraph
- **Logic**: Implements an iterative algorithm that removes the minimum degree vertex at each step, tracking density.

3. Main Workflow

The program follows these steps:

1. Parse command-line arguments (input file, output directory, sampling options)
2. Load and preprocess the Amazon metadata
3. Build the co-purchasing network graph
4. Perform various network analyses:
    - Compute basic network metrics
    - Calculate degree distribution
    - Detect communities
    - Analyze cross-category connections
    - Calculate centrality measures
    - Find the densest subgraph
5. Save results to files for visualization and further analysis

D. Tests

The project includes comprehensive tests for all major components:

```
$ cargo test
running 8 tests
test test_graph_building ... ok
test test_network_metrics ... ok
test test_degree_distribution ... ok
test test_community_detection ... ok
test test_cross_category_connections ... ok
test test_centrality_measures ... ok
test test_densest_subgraph ... ok
test test_extract_subgraph ... ok

test result: ok. 8 passed; 0 failed; 0 ignored; 0 measured; 0
filtered out
```

- test_graph_building: Verifies that the graph correctly represents product relationships
- test_network_metrics: Ensures metrics calculations are accurate
- test_degree_distribution: Checks degree distribution calculation logic
- test_community_detection: Validates community detection functionality

- test_cross_category_connections: Ensures cross-category analysis produces expected results
- test_centrality_measures: Verifies centrality calculations
- test_densest_subgraph: Tests the densest subgraph algorithm
- test_extract_subgraph: Validates subgraph extraction functionality

These tests ensure the reliability of the analysis pipeline and correctness of the algorithms.

E. Results

Network Structure

The analysis of the Amazon co-purchasing network by sampling 10,000 data revealed several key insights:

- **Network Metrics**:
    - Total nodes (products): 10,000
    - Total edges (co-purchases): 30,007
    - Network density: 0.0006, which indicates a very sparse structure as expected in large-scale co-purchasing network
    - Connected components: 1 - the sample form a single connected component
    - Average degree: 6.00
    - Max degree: 16
    - Average clustering coefficient: 0.50 - moderate local clustering of products
- **Degree Distribution**: The network follows a power-law degree distribution, with most products having few connections and a small number of products being highly connected.
    - Most products have between 5-12 connections
    - A few highly connected products reach up to 16 co-purchase links
- **Communities**:
    - The algorithm detected 1 main community of size 8,834 which suggests strong interconnectedness across product types in this sample
    - The top 5 categories in the community highlight product overlap especially across
        - Books (1988)
        - Clothing (1969)
        - Music (2,064)
        - Electronics (1984)
        - Movie & TV (1995)
- **Cross-Category Connections**: The strongest cross-category relationships were found between:
    - Clothing → Movies & TV (2,464 connections)
    - Movies & TV → Music (2,462 connections)
    - Books → Music (2,432 connections)
    - Clothing → Music (2,426 connections)
    - Books → Movies & TV (2,422 connections)

These connections reveal opportunities for cross-selling and indicate consumer preference patterns or consumer lifestyle trends.

- **Central Products**: Several products emerge as particularly central to the network, functioning as bridges between different product communities.

ASIN	Title	Category	Degree	Betweenness	Closeness
A00006879	Product 6879	Clothing	16	0.1987	0.2056
A00003827	Product 3827	Books	14	0.3135	0.2043
A00001566	Product 1566	Clothing	14	0.3828	0.1988
A00007717	Product 7717	Music	13	0.9368	0.1985
A00005295	Product 5295	Electronics	13	0.6898	0.2010

- **Densest Subgraph**:
  - Contains 8,834 products
  - Density: ~0.0006, which implies the densest subgraph algorithm didn't find a tightly packed region, but rather retained the whole component

F. Usage Instructions

Building the Project

```
git clone [your-repo-url]
cd amazon-network-analysis

cargo build --release
```

Running the Analysis

bash

```
./target/release/amazon_network_analysis --sample

# Run with custom parameters
```

```
./target/release/amazon_network_analysis \ --input-file
data/amazon-meta.txt \ --output-dir results \ --num-communities 10 \
--min-community-size 3 \ --sample-size 10000
```

Command-line Arguments

- `--input-file`: Path to the Amazon metadata file (default: "data/amazon-meta.txt")
- `--output-dir`: Directory to save results (default: "results")
- `--num-communities`: Number of communities to detect (default: 10)
- `--min-community-size`: Minimum number of nodes in a community (default: 3)
- `--sample`: Run on a sample of the data instead of the full dataset
- `--sample-size`: Size of the sample (default: 10000)

Expected Runtime

- Sample run (10,000 products): ~`1.5 minutes
- Full dataset (~500,000 products): ~45 minutes

G. AI-Assistance Disclosure and Citations

During the development of this project, I utilized Claude AI to assist with the following aspects:

1. **Project Structure Planning**: I used Claude to help design the modular architecture and identify appropriate data structures for the analysis. I'm particularly proud of how Claude helped me organize the codebase in a clean, maintainable way that separates concerns effectively.
2. **Rust-Specific Implementation Help**: As I'm still learning Rust, I sought assistance with language-specific constructs, particularly around the use of the petgraph library and proper error handling.
3. **Algorithm Implementation**: For the densest subgraph algorithm, I used AI to help implement the 2-approximation algorithm based on the paper mentioned in the project description. The AI provided the following pseudocode which I adapted to Rust:

```
1. Start with the entire graph
2. Iteratively remove the vertex with minimum degree
3. Track the density at each step
4. Return the subgraph with maximum density
```

All code was reviewed and understood before implementation, and I made modifications to ensure it worked correctly with our specific data structures.

Other References

- Stanford SNAP datasets: https://snap.stanford.edu/data/
- petgraph documentation: https://docs.rs/petgraph/
- "Defining and Evaluating Network Communities based on Ground-truth" by J. Yang and J. Leskovec